Generic Partial Decryption as Feature Engineering for Neural Distinguishers

Emanuele Bellini[1 [0000−0002−2349−0247]], Rocco Brunelli[2 [0009−0005−2481−410X]], David Gerault[1 [0000−0001−8583−0668]], Anna Hambitzer[1 [0000−0002−5357−832X]], and Marco Pedicini[2 [0000−0002−9016−074X]]

[1] Technology Innovation Institute, Abu Dhabi, UAE
{name.lastname}@tii.ae
[2] Roma Tre University, Rome, Italy
{name.lastname}@uniroma3.it

Abstract. In Neural Cryptanalysis, a deep neural network is trained as a cryptographic distinguisher between pairs of ciphertexts $(F(X), F(X \oplus \delta))$, where F is either a random permutation or a block cipher, δ is a fixed difference. The AutoND framework aims to use neural distinguishers that are treated as a generic tool and discourages cipher-specific optimizations. On the other hand, works such as [LLS+24] obtain superior distinguishers by adding dedicated features, such as selected parts of the difference in the previous rounds, to the input of the neural distinguishers. In this paper, we study **Generic Partial Decryption** as a feature engineering technique and integrate it within a fully automated pipeline, where we evaluate its effect independently of the number of pairs per sample, with which feature engineering is often combined. We show that this technique matches state-of-the-art dedicated approaches on Simon and Simeck. Additionally, we apply it to Aradi, and present a practical neural-assisted key recovery for 5 rounds, as well as a 7-rounds key recovery with 2^{70} time complexity. Additionally, we derive useful information from the neural distinguishers and propose a non-neural version of our 5-round key recovery.

Keywords: Neural Cryptanalysis · Differential Cryptanalysis · Block Cipher · Partial Decryption · Simon · Simeck · Aradi

1 Introduction

Block ciphers are a cornerstone of modern cryptography, and secure block ciphers can be used as a building block for many symmetric key primitives, making their analysis one of the most fundamental problems in symmetric cryptanalysis.

Differential Cryptanalysis. Differential cryptanalysis, the study of how modifications of the inputs propagate to the output, is one of the most important techniques for establishing bounds on the resistance of block ciphers. While it was first publicly presented by Biham and Shamir in 1990 [BS91], it is believed

to have been known and used as early as 1974 [Cop94]. Despite 50 years of research, new insights on differential cryptanalysis are still presented regularly at major security conferences. For instance, a new generic differential key recovery algorithm was presented at Eurocrypt 2024 [BDD+24]. These advances often go hand in hand with the improvement of tools. In particular, the search for differential characteristics has been made significantly easier in recent years through automatic search tools based on SAT, SMT, MILP, or CP. The ability to easily obtain, analyze, or count differential characteristics with given properties lets the cryptographer focus on novel analysis techniques rather than the tedious implementation of dedicated search algorithms. However, difficult instances keep challenging these modern tools despite the constant evolution of techniques [SWW21]; as a result, finding accurate differential bounds is still an open problem for many primitives.

These challenges become even more acute when investigating variants of differential cryptanalysis, such as multiple differential cryptanalysis [BG11], where the cryptographer considers not one but many of the possible output differences. To this day, it is extremely difficult to evaluate the resistance of a block cipher to such attacks.

Neural Distinguishers. At CRYPTO 2019, A. Gohr proposed to use deep learning in the context of differential cryptanalysis [Goh19]. In this work, *neural distinguishers* are trained to distinguish the Speck 32 [BSS+13] encryption of two plaintexts related by a fixed input difference from random pairs. The author shows that a differential distinguisher using the entire (under the Markov assumption) Differential Distribution Table (DDT) of the cipher obtains very close performance, hinting that neural distinguishers rely on multiple-differential properties. However, computing the entire DDT is usually not tractable when analyzing primitives with states larger than 32 bits. To that extent, neural distinguishers can be seen as a very potent tool to assess the resistance of a cipher to multiple differential cryptanalysis, filling a gap in automatic tools. For instance, for Speck 32, the best 7-rounds differential characteristics have at best probability 2^{-19} [LLJW21], but neural distinguishers can distinguish a single pair from random with 60% accuracy, indicating stronger multiple-differential properties than expected.

Unfortunately, neural distinguishers, like most deep-learning techniques, suffer from explainability issues. We can empirically observe that a neural distinguisher seems to be able to learn some form of the compressed representation of the DDT, but building such a compressed representation without deep learning is an open problem. Nevertheless, in the absence of a better tool, neural distinguishers can be used to improve our understanding of multiple differential properties and develop new attacks.

Automation. In that spirit, at FSE 2024, the AutoND [BGH+23] framework was introduced. This framework aims at automatically building a neural distinguisher from the implementation of a cipher, providing a bound on the number of rounds for which strong multiple differential properties exist. The au-

thors present the tool as akin to libraries such as Tagada [LDLS21], CASCADA [RR22] or CLAASP [BGG+24], that, given a cipher implementation, build distinguishers. Such tools allow cryptographers to focus on developing new techniques rather than re-implementing already existing methodologies to find distinguishers. Similarly, training a neural distinguisher from scratch is tedious, and many recent works focused on optimizing the accuracy as much as possible through lengthy hyper-parameters tuning, custom feature engineering, or dedicated curriculum learning procedures, when the existence of a neural distinguisher for a given number of rounds is, on its own, valuable information.

The initial version of AutoND matches most of the current literature despite being completely generic and requiring no input from the cryptographer other than the cipher implementation. On the other hand, dedicated approaches using primitive-specific features can result in better distinguishers. For example, in Simon, given two ciphertexts, their full difference at the previous round can be inferred, along with probabilistic information on the difference two rounds before. Adding this information to the input of the neural distinguisher can significantly improve its accuracy [LLS+24].

It is natural that such dedicated feature engineering provides better distinguishers in comparison to the typical black-box approach to neural cryptanalysis. For instance, [Dom12] emphasizes that the selection and engineering of features often have a greater impact on model performance than the choice of algorithm: "feature engineering is more difficult because it is domain-specific while learners can be largely general purpose" (*Communications of the ACM*, 2012).

Our goal in this research is to fill this gap in a generic way. More specifically, we add decryption with arbitrary keys to the features used in the AutoND framework and show that this is sufficient to match the state of the art. Using this approach, we preserve deterministic structural properties of the cipher without needing to specifically isolate the relevant ones. Concretely, this partial decryption naturally contains some amount of irrelevant information (the non-deterministic propagations), but it appears to be easy for the neural distinguisher to filter out useless information. We test this hypothesis on the Simon, and Simeck ciphers.

Aradi as a Case Study. The block cipher Aradi [GMW24] was published by NSA researchers without a security analysis, and very few public independent analyses have been performed (e.g. [BRRT24, BFG+24a, ADG24]). We apply our framework to Aradi and present the first practical neural key recovery attack on 5 rounds, as well its classical equivalent.

1.1 Our Contributions

1. We introduce the automated Generic Partial Decryption pipeline (Section 4), and obtain competitive neural distinguishers for Simon, Aradi, and Simeck. Our multi-pair neural distinguishers cover 12 rounds for both Simon and Simeck with an accuracy of 0.5156, slightly surpassing the state-of-the-art

results reported in [LLS+24]. Generic Partial Decryption significantly improves the 5-round accuracy for Aradi, increasing it from approximately 60% to 77% in the single-pair setting, and 100% when in the multi-pair setting.

2. We enhance the explainability of previous results using partial decryption by comparing the outcomes across four configurations: *i)* using a simple ciphertext pair as a sample, *ii)* incorporating multiple ciphertext pairs per sample, *iii)* increasing the amount of training data, and *iv)* applying partial decryption. This approach enables a clear differentiation of the individual contributions to the overall accuracy of the neural distinguisher. The results are reported in Section 5.
   For Simon and Simeck, the primary accuracy improvements stem from employing multi-pair neural distinguishers *(ii)* and using more training data *(iii)*, while the contribution of partial decryption *(iv)* is comparatively smaller.
   In contrast, for Aradi, the accuracy improvement due to partial decryption *(iv)* is of a similar magnitude to the gains achieved through multi-pair usage *(ii)*.
3. Using our 4-rounds neural distinguishers, we present the first practical neural key recovery attack on 5 rounds of Aradi, with data complexity 2^9 (for 75% success rate), and time complexity dominated by 2^{40} neural distinguisher evaluations. We extend this attack to 7 rounds, albeit with non-practical time complexity. More importantly, for the first time, we leverage information learned from the neural distinguisher to build a classical key recovery attack. For 5 rounds, this attack has data complexity comparable to the neural one, and time complexity 2^{42}. These attacks are discussed in Section 6.

2 Preliminaries

In this paper, we use the standard symbols for the boolean operations: $\oplus$ XOR, $\lll$ (resp. $\ggg$) α for left (resp. right) rotation by α bits, $\odot$ for bitwise AND, $||$ for concatenation, and $\boxplus$ and $\boxminus$ for modular addition and subtraction.

2.1 Analyzed Ciphers

Simon and Simeck

Simon [BSS+13] and Simeck [YZS+15] are families of lightweight block ciphers, where the state $l||r$ is updated using the round key k_i as follows:

$$l_{i+1}||r_{i+1} \leftarrow ((l_{i-1} \lll \alpha) \odot (l_{i-1} \lll \beta)) \oplus (l_{i-1} \lll \gamma) \oplus r_{i-1} \oplus k_i||l_{i-1}$$

We focus on the variants using a 32-bit state and a 64-bit key, Simon 32/64 and Simeck 32/64, using rotational constants $\alpha = 1$, $\beta = 8$ and $\gamma = 2$ for Simon 32/64, $\alpha = 0$, $\beta = 5$ and $\gamma = 1$ for Simeck 32/64.

Aradi

ARADI [GMW24] is a substitution-permutation network (SPN), taking as input a 128-bit block and a 256-bit key designed for low-latency applications. The round function updates the state $(w||x||y||z)$ represented as four 32-bit words through a column-wise 4-bit S-box π followed by a row-wise linear layer Λ_i :

$$\Lambda_i(w||x||y||z) := (L_i(w)||L_i(x)||L_i(y)||L_i(z)),$$

$$L_i(s) = L_i(s_l; s_r) := (s_l \oplus (s_l \lll \alpha_i) \oplus (s_r \lll \gamma_i)||s_r \oplus (s_r \lll \alpha_i) \oplus (s_l \lll \beta_i))$$

If we denote by $\tau_v(s)$, the bitwise XOR of s with v, then the ARADI encryption function has the form:

$$\tau_{k^{16}} \circ (\Lambda_{15} \circ \pi \circ \tau_{k^{15}}) \circ \cdots \circ (\Lambda_1 \circ \pi \circ \tau_{k^1}) \circ (\Lambda_0 \circ \pi \circ \tau_{k^0})$$

where k^i denotes the round key, we consider it composed of four words of 32-bits $k^i = k^i_w||k^i_x||k^i_y||k^i_z$ corresponding to the four words of the block.

2.2 (Neural) Differential Cryptanalysis

Differential cryptanalysis [BS91] is a chosen plaintext attack in which an attacker analyzes how an injected difference δ in a plaintext pair $(P, P' = P \oplus \delta)$ propagates into the resulting ciphertext pair (C, C'). In differential cryptanalysis, the cryptanalyst is interested in finding differentials δ, γ such that the equation $F(P) \oplus F(P \oplus \delta) = \gamma$ holds for many plaintexts P for a primitive F.

Neural Differential Cryptanalysis. Neural differential cryptanalysis was introduced at CRYPTO 2019 by Gohr [Goh19]. In this seminal work, a *neural distinguisher* is trained to classify pairs of ciphertexts as *real* or *random*. The real pairs correspond to the SPECK 32/64 encryption of plaintexts P and $P \oplus \delta$, where $\delta =$ `0x400000` is chosen for its favorable diffusion properties, whereas the random pairs are sampled uniformly at random.

A neural distinguisher is a neural network whose architecture is tuned for cipher analysis. Concretely, it is a parametrized function f_θ composed of affine layers followed by non-linear activations, mapping the concatenated pair $(C_0 \parallel (C_0 + \delta))$ to a single score. During *supervised training* the network receives labelled examples (x, y) with $y \in \{0, 1\}$ indicating the correct class and iteratively updates θ via stochastic-gradient optimization to minimize a differentiable loss. Regularization techniques mitigate *overfitting* – memorizing the training set – thereby promoting generalization to previously unseen data. The resulting *accuracy* is evaluated on a separate test set and measures how reliably the trained model distinguishes truly random data from encrypted pairs for the chosen input difference δ. Gohr reports test accuracies exceeding 92% for 5 rounds of SPECK, falling to roughly 52% for 8 rounds, illustrating both the potential and the current limitations of neural distinguishers.

Among the almost 200 papers citing this seminal work, many investigate how to generalize it to other primitives, often using *feature engineering*, where the cryptographer uses his knowledge of the cipher's structure to craft features that may be difficult for the $\mathcal{ND}$ to learn on its own.

2.3 Feature Engineering

Feature engineering is a fundamental technique in machine learning, focusing transforming raw data into a more suitable set of features, for instance, through normalization, encoding, or aggregation, to improve the interpretability and accuracy of the model. In *"Feature Engineering for Machine Learning"* [ZC18], a *feature* is loosely described as "a numeric representation of raw data".

3 Related Work

Neural Distinguisher Works with Focus on Feature Aggregation. It is commonplace that distinguishers having access to more data tend to perform better; therefore, multi-pair distinguishers, which are trained on samples containing more than two ciphertexts ($m > 1$), have been used in works such as [BGPT21a] ($m = 20, 100$), [CSYY22] ($m = 8$), [HRCF21] ($m = 64, 128$), and [LLS$^+$24] ($m = 8, 16$).

However, the use of multi-pair samples has been criticized in [GLN23], where a *combined response score* is defined to aggregate the scores produced by a single-pair distinguisher on multiple samples sharing the same label. Using this technique, the authors compare the accuracies of multi-pair ($m > 2$) distinguishers with the combination of m single-pair distinguisher scores, and conclude that "the claimed improvement [of multipair-distinguishers] is at best non-existent in most cases". This conclusion relies on a *combined response score*, which aggregates the scores produced by a single-pair distinguisher on multiple samples sharing the same label.

Depending on the strictness of the feature engineering definition, one may argue that combining multiple samples with the same label into a single sample does not constitute "feature engineering". This process does not involve modifying or generating new features but rather aggregates existing data points.

Neural Distinguisher Works with Focus on $\oplus$-Feature Engineering. In differential cryptanalysis, the ciphertext difference $C_0 \oplus C_1$ is the main feature. Many studies replace concatenated ciphertext pairs $(C_0||C_1)$ by their XOR difference $(C_0 \oplus C_1)$, such as [BBCD21], Hou *et al.* [HRCF21], and Yadav *et al.* [YK21].

Using features based on $C_0 \oplus C_1$ can accelerate training due to the reduced input length, which simplifies parts of the training process. However, this approach may sacrifice some information, often resulting in distinguishers with lower accuracy compared to those trained on $C_0||C_1$. This trade-off underscores that feature engineering is closely linked to explainability; deriving optimal input features is most effective when the learned properties of the neural network are well understood. Benamira *et al.* [BGPT21a] examined the properties of pairs that were correctly classified, suggesting that Gohr's neural distinguishers learn differential-linear features.

Neural Distinguisher Works with Focus on Feature Engineering by Partial Decryption. It is often possible to derive truncated information on the difference in the previous rounds of a cipher. This additional information, treated as an additional feature, is trivial given the algorithm of the primitive, but can be arbitrarily hard in the black-box scenario (e.g., for unkeyed cryptographic permutations). In [BGPT21a], the authors use such partial information in an explainability study. In [BGL+22] and [LLS+24], the authors improve the accuracy of neural distinguishers on SIMON through similar techniques, using inferred information from one and two rounds ahead.

The authors of [BGPT21a] propose the hypothesis that the $\mathcal{ND}$ is able to infer information on the difference from 2 rounds ahead; such feature engineering connects naturally with that hypothesis. Based on this observation, Liu et al. [LRCL23] aim to use partial decryption to improve neural distinguishers: they use randomly generated subkeys to decrypt one round and develop new data formats MRMSP (Multiple Rounds Multiple Splicing Pairs) and MSMSD (Multiple Rounds Multiple Splicing Differences).

Works with Focus on Generic Tools and Automation. Recent advancements have focused on developing generic tools and automation for cryptanalysis. Open-source cryptographic libraries, such as TAGADA [LDLS21], Cascada [RR22], and CLAASP [BGG+24], aim to enable fully automated cipher analysis, providing frameworks for analyzing a variety of cryptographic primitives.

Gohr, Leander, and Neumann [GLN23] advocate for the use of neural distinguishers as a generic tool comparable to well-established methods like SAT and MILP solvers. Their work emphasizes the potential of neural distinguishers to provide automated, flexible, and efficient solutions to cryptanalytic problems.

At FSE 2024, Bellini *et al.* [BGH+23] introduced AutoND, a cipher-agnostic neural distinguisher pipeline. AutoND achieves state-of-the-art results across various cryptographic primitives without requiring human intervention, demonstrating the feasibility of automated pipelines for neural-based cryptanalysis.

4 Design of the Generic Partial Decryption Pipeline

To design our Generic Partial Decryption pipeline, we begin by outlining our strategy for generic partial decryption in Section 4.1. This strategy enables the derivation of various potential data formats for the neural distinguisher. From these, we select four data formats for detailed investigation, as described in Section 4.2.

After determining the input data format, the next critical decisions involve selecting the input difference (Section 4.3) and choosing the appropriate neural network architecture (Section 4.4). Furthermore, we aim to simplify the complex training pipelines commonly employed in prior works by reducing the amount of training data required, leading to our streamlined training pipeline design (Section 4.5).

To address concerns regarding the use of multiple pairs, we choose to train single-pair distinguishers, which can also be evaluated on multiple pairs for comparison with existing literature, as detailed in Section 4.6.

4.1 Our Strategy for Generic Partial Decryption

The specific features utilized by the $\mathcal{ND}$ to make predictions remain largely unknown. It has been established that these features are mostly differential in nature, but a small non-differential component is shown in [Goh19], and the presence of differential-linear features has been suggested in [BGPT21a]. Various improvements, such as designing new $\mathcal{ND}$ architectures or altering the input data format, have been proposed to enhance $\mathcal{ND}$ performance. However, these enhancements are typically tailored to specific ciphers or those sharing particular characteristics, complicating the understanding of how and why the $\mathcal{ND}$ models the cipher characteristics effectively.

In [Goh19], the key recovery strategy uses the response profile of the $\mathcal{ND}$ when the last round is decrypted with an incorrect key. In [BGPT21a], the authors use a similar technique to build equivalent classical distinguishers. In addition, they observe that the right part of the Speck 32 state at the previous round can be computed without knowing the key and propose to use it as an additional feature for the neural distinguisher. In [BGL+22], the authors extend that notion to the Simon 32/64 cipher family and compare different types of related feature engineering. In [LLS+24], the authors consider not only such deterministic features but also probabilistic ones. Notably, for the Simon-like ciphers, they build features based on the (deterministically obtained) difference at round $i-1$ and an approximation of the difference at round $i-2$, obtained by decrypting the last 2 rounds with subkey 0.

According to [LLS+24], this approach significantly improves accuracy and opens avenues for further extensions. They also design $\mathcal{ND}$ architectures specifically suited for targeted tasks.

Our primary goal is to develop a general understanding of the features captured by the $\mathcal{ND}$ to enhance interpretability. Building on the work in [LLS+24], analyzing the differentials in earlier rounds could yield better input features for training $\mathcal{ND}$ models. However, if partial decryption is limited to the specific cipher under analysis, it demands significant manual effort from the cryptanalyst, thereby reducing the suitability of the $\mathcal{ND}$ as a generic cryptanalytic tool.

The propagation of a differential through a cipher typically gains more entropy at each round until full diffusion is achieved. Therefore, the difference at round $i-1$ normally holds more useful information than the difference at round i for a distinguisher. Given knowledge of the round function of a cipher, it is often possible to derive partial information about the difference in the previous rounds. For Feistel-like ciphers such as Simon and Speck, part of the previous round state can be directly obtained. For SPNs, such as Aradi, truncated differential information on the previous round can be recovered (in particular, the activity pattern of the S-Boxes). In addition, high probability differential tran-

sitions, by definition, hold for a large portion of the key space so that partial decryption with an incorrect key has a high chance of preserving them.

While such information is trivial for the cryptographer to obtain, it would be unreasonable to expect the neural distinguisher to learn it on its own. For instance, in the case of SIMON, learning the round function to retrieve the previous round difference appears difficult for a neural distinguisher. By construction of Feistel-like ciphers, it is trivial for a cryptographer to retrieve information on the previous round difference; on the other hand, if the round function is sufficiently complex, it becomes arbitrarily hard for a neural network with black-box access to do so.

A cryptographer's natural approach would be to observe which parts of the previous rounds' differences can be recovered with a high probability and use them only. This is also the approach taken by [LLS+24]. On the other hand, following the goal to limit human input to the mere specification of the cipher, we would like to derive such information automatically in a generic fashion. More specifically, we provide partial decryption with a random key as a feature and let the neural distinguisher find the relevant parts and discard the less relevant ones. Without loss of generality, we use the subkey 0 in our experiments rather than a random one; intuitively, the exact choice has no impact as long as the expected Hamming distance with the actual key is $\frac{n}{2}$.

The number of rounds for which such partial decryption holds information varies depending on the cipher under consideration; for instance, in [GLN22a], the authors use information from many previous rounds in their analysis of KATAN. In this paper, for the sake of confirming the benefit of partial decryption as a feature, we restrict ourselves to 2 rounds of partial decryption. This captures the relevant information for most block ciphers while keeping the size of the dataset and experimental time reasonable. We expect our conclusions to hold when including the partially decrypted difference in all rounds, but it may scale poorly for specific examples such as KATAN, where dozens of rounds would make the sample size unpractical.

We extend the concept of *Partial Decryption*, originally tailored to a specific cryptographic primitive, to the broader framework of *Generic Partial Decryption*. This approach is formalized as follows:

Definition 1. *Let $\mathcal{P}$ the plaintext set, $\mathcal{C}$ the ciphertext set, $\mathcal{K}$ the key space, f the encryption round function and g the decryption round function such that:*

$$f : \mathcal{P} \times \mathcal{K} \to \mathcal{C}, \quad f_k^i(C^{i-1}) := f(k_i, C^{i-1}) = C^i$$
$$g : \mathcal{C} \times \mathcal{K} \to \mathcal{P}, \quad g_k^i(C^i) := g(k_i, C^i) = C^{i-1}.$$

where k_i is the i-th round key, derived from the master key $k \in \mathcal{K}$.
Then, we can define the encryption function *and the* decryption function *as:*

$$E_k^n(C^0) = f_k^{n-1} \circ \cdots \circ f_k^0(C^0) = C^n, \quad D_k^n(C^n) = g_k^0 \circ \cdots \circ g_k^{n-1}(C^n) = C^0,$$

such that:

$$f_k^i \circ g_k^i(C) = C \quad \textit{and} \quad g_k^i \circ f_k^i(P) = P \quad \textit{with } i = 1, \ldots, n.$$

Definition 2. *Let* $(C^i, (C')^i) = (E_k^i(P), E_k^i(P'))$ *be a pair of the encryption function outputs after* i *rounds. Then, the* differential *at the* i*-th round is:*

$$\Delta^i = E_k^i(P) \oplus E_k^i(P') = C^i \oplus (C')^i.$$

We call the ***1-Partial Differential*** *at round* $i-1$ *and the* ***2-Partial Differential*** *at round* $i-2$*:*

$$\tilde{\Delta}^{i-1} = g(0, C^i) \oplus g(0, (C')^i), \quad \tilde{\Delta}^{i-2} = g\left(0, g(0, C^i)\right) \oplus g\left(0, g(0, (C')^i)\right) \quad (1)$$

where 0 is the round key (i.e. $k_i = 0$*). More in general, the* k***-Partial Differential*** *is:*

$$\tilde{\Delta}^{i-k} = \underbrace{g(0, g(0, \ldots, g(0, C^i))}_{k-times} \oplus \underbrace{g(0, g(0, \ldots, g(0, (C')^i))}_{k-times}.$$

In the Generic Partial Decryption technique, the ciphertexts are decrypted with round key 0. In contrast to the approach in [LLS+24], we do not select which parts of the resulting words are kept. For the remainder of this paper, we focus exclusively on the application of $\tilde{\Delta}^{i-1}$ and $\tilde{\Delta}^{i-2}$.

4.2 Choice of the Data Format

Based on our Generic Partial Decryption strategy, several choices of data formats are possible. We focus on three specific formats:

Given an input difference δ and a pair of plaintexts $(P, P \oplus \delta)$, we denote ciphertext pairs encrypted with the same key as $(C, C') = (E_k^r(P), E_k^r(P \oplus \delta))$ for some $k \in \mathcal{K}$.

Simple Pair (SP). The first data format, SP, is the most commonly used in the literature; It provides a baseline to evaluate whether adding additional features can improve training:

$$(C^r, (C')^r).$$

Last Differential, Simple Pair, 1-Partial Differential and 2-Partial Differential (LD, SP, 1PD, 2PD). The second format, (LD, SP, 1PD, 2PD), is closely related to options recently investigated in [LLS+24] and incorporates both ciphertext pairs and partial differentials:

$$(\Delta^r, C^r, (C')^r, \tilde{\Delta}^{r-1}, \tilde{\Delta}^{r-2}).$$

Last Differential, 1-Partial Differential and 2-Partial Differential (LD, 1PD, 2PD). The third format (LD, 1PD, 2PD) is unique in that it does not utilize ciphertext pairs, relying solely on differential values. This format serves to evaluate the influence of non-differential features on the accuracy of the $\mathcal{ND}$:

$$(\Delta^r, \tilde{\Delta}^{r-1}, \tilde{\Delta}^{r-2}).$$

It is commonly assumed, following the conclusions of [Goh19], that the ciphertexts themselves are needed to reach optimal accuracy; our last format (LD, 1PD, 2PD) aims at testing this hypothesis.

4.3 Choice of Input Differences

The choice of the input difference used in a neural distinguisher has been shown to have a significant impact on the number of distinguished rounds and the accuracy [BGPT21b]. A good input difference for a neural distinguisher is not necessarily the input difference for the best classical differential trail. It suffices to observe that differential trails for a few rounds have relatively low probability (*e.g.,* 2^{-9} for 5 rounds of Speck), whereas neural distinguishers distinguish based on a single pair. Therefore, the property that they identify has to occur with significantly higher probability than a classical trail, and properties such as multiple differentials and differential-linear play an important role.

It is commonplace in neural distinguishers research to pick an input difference with a low Hamming Weight, as it is expected to propagate slowly and enable longer neural distinguishers. These input differences are often chosen from intermediate rounds of known trails for the cipher under study, as is the case in [LLS+24].

At FSE 2024, the authors of [BGH+23] introduced an evolutionary optimizer for identifying suitable input differences for neural distinguishers in a generic manner. This optimizer operates on the premise that neural distinguishers detect truncated differentials from earlier rounds [BGPT21b]. Given this approach, the existing evolutionary optimizer appears well-suited for application to partially decrypted input data formats.

4.4 Choice of the Neural Network Architecture

For a comprehensive overview of the neural network architectures for cryptanalysis, we refer readers to the recent systematization of knowledge [GHHP24]. To motivate our choice of neural network architecture, we provide the following summary: Multi-layer perceptrons (MLPs) have been widely explored as a basic yet lightweight architecture, for example in [BR21, ZZY+21, ERP22]. While MLPs are efficient and easy to implement, their performance is often surpassed by more advanced architectures. The majority of subsequent works build upon Gohr's original neural network design, which has been adapted and refined in various studies, among others [SZM21, BBCD22, WTZ+22]. Although Gohr's network can be adapted to new ciphers, this process necessitates careful tuning of numerous hyper-parameters [GLN23]. Advanced components, such as attention mechanisms [DCC23], LSTMs [BBCD22, SSL+22], GoogLeNet-inspired Inception modules [ZWC23, ZLWL23, BLYZ23], and DenseNets [SM23], have also been explored. While these approaches demonstrate strong results for specific ciphers, they are often highly cipher-specific and are accompanied by significantly increased training times.

At FSE 2024, DBitNet was introduced as a "cipher-agnostic" architecture [BGH+23]. This architecture avoids cipher-specific components, employs a simplistic design, and achieves state-of-the-art results across various primitives without requiring any modifications or hyper-parameter adjustments. Given our goal of developing generic applicability, we focus on two specific architectures:

DBitNet, which emphasizes generalization across ciphers, and GohrAMS, a variant of Gohr's network that incorporates AMSGrad learning rate adaptation also introduced in [BGH+23].

The Neural Networks we consider are Gohr's original depth-1 network [Goh19] with AMSGrad learning rate GohrAMS and DBitNet [BGH+23]. We will briefly describe them and refer to their respective publications for more details.

GohrAMS. The Neural Network proposed in [Goh19] is composed of a preprocessing layer in which the input dimension is reshaped, followed by three main blocks: one slicing Convolutional Layer, n iterations of a Residual Block, and a Prediction layer. We use the AMSGRAD optimizer [RKK19] instead of the original learning rate scheduler, as it was shown in [BGH+23] to achieve equivalent performance without the need for manual fine-tuning of the original cyclic learning rate schedule.

DBitNet. DBitNet employs dilated convolutional layers to effectively combine long-range and short-range dependencies across consecutive convolutional layers. The architecture begins with a dilated convolutional layer that adapts to the input size, with an initial dilation rate of $dr_0 = \left\lfloor \frac{\text{input size}}{2} - 1 \right\rfloor$. This is followed by a layer targeting local interactions with a fixed dilation rate of 2. Each dilated convolutional layer reduces the neuronal width by approximately half. The alternation between dilated convolution and standard convolution of neighboring bits continues for $dr_i = \left\lfloor \frac{r_{i-1}+1}{2} - 1 \right\rfloor$, with $i = 1, \ldots, \lfloor \log_2(\text{input size}) - 3 \rfloor$.

As the neuronal width decreases with each dilated step, the number of filters increases incrementally to compensate for the reduced spatial dimensions. This design ensures a balance between dimensionality reduction and feature extraction, enabling the network to achieve effective representation and generalization across a variety of ciphers.

4.5 Choice of the Training Pipeline

The training pipeline is known to be determinant when targeting higher rounds, for which the signal of the distinguisher becomes weaker. In [Goh19], the 8-round distinguisher was obtained by retraining the best distinguishers starting from likely differences in the middle rounds and increasing the amount of training data by a factor of 100. Similarly, in [LLS+24], the 12-round Simon 32/64 distinguisher is built by iteratively retraining the best distinguishers on curated data, with a dedicated cyclic learning rate. The authors adopt a similar pipeline for Simeck 32/64.

We aim to achieve similar accuracies without using such dedicated techniques. In particular, we improve the (already competitive) results we obtain with a simple training pipeline by applying the simple polishing pipeline defined in [BGH+23].

4.6 Choice of Single- or Multi-Pair Distinguisher

The [LLS+24]'s $\mathcal{ND}$s are multi-pair Distinguishers trained with samples composed of 8 pairs, increasing the amount of data required for training and testing. On the other hand, [GLN22b] proposes a method to create a multi-pair distinguisher from a single-pair distinguisher. The algorithm, described below Equation 2, takes as input m ciphertext pairs, either all real or all random, but otherwise assumed to be independent. The $\mathcal{ND}$ is assumed to be perfectly calibrated so that the pairwise scores $ND(c_i) = p_i$ can be interpreted as probabilities. Under these assumptions, the Multi-Pair distinguisher derives the probabilities $P_{real} = \mathbb{P}[\text{all real} \mid \text{observed scores}]$ and $P_{rand} = \mathbb{P}[\text{all random} \mid \text{observed scores}]$, aggregates them as the combined response score:

$$\frac{P_{real}}{P_{real} + P_{rand}} = \frac{1}{1 + \frac{P_{rand}}{P_{real}}} = \frac{1}{1 + \prod_{i=1}^{m} \frac{1-p_i}{p_i}}. \tag{2}$$

We use Equation 2 following [GLN22b]: (i) We validate our $\mathcal{ND}$ for each format on 10^6 pairs; these datasets are denoted $(X_j)_j$. (ii) We split the dataset according the labels; (X_j^1) are the real samples and (X_j^0) the random ones. Let $N^t = \#(X_j^t)$, be the cardinality of the set for $t \in \{0, 1\}$. (iii) We obtain the respective scores of the samples as $p_j^t = ND(X_j^t)$ with $t \in \{0, 1\}$. (iv) We divide the corresponding (p_j^t) into m subgroups and apply Equation 2. We obtain in total $M^t = \lfloor N^t/m \rfloor$ prediction for $t \in \{0, 1\}$. (v) Each M^t subsample is classified based on its combined response score. (vi) The accuracy is given from the ratio of the right-labeled subsamples over the total subsamples we have.

4.7 Final Pipeline for Generic Partial Decryption

Based on the previously presented considerations, we arrive at the following pipeline for generic partial decryption: The main algorithm trains a neural distinguisher $\mathcal{ND}$ to identify the highest non-random round of a block cipher implementation $E_k(\cdot)$. It adopts a curriculum learning approach, beginning from an initial round r_0 and progressively training the $\mathcal{ND}$ on datasets generated for increasing rounds. The training process continues until the validation accuracy drops below a predefined threshold (ten standard deviations (10σ) above the random accuracy of fifty percent).[3] Datasets for each round are created using the chosen data format in the GeneratePDDataset algorithm.

The GeneratePDDataset sub-algorithm constructs a labeled dataset $\mathcal{D}$ for a given round r. It generates N samples through the following steps: (i) Randomly select plaintexts $P \in \mathcal{P}$ and keys $k \in \mathcal{K}$. (ii) Assign a random label $y \in \{0, 1\}$. (iii) Compute ciphertext pairs (C, C') using the r-round encryption function $E_k^r(\cdot)$ on $(P, P \oplus \delta_0)$. (iv) C, C' becomes a random pair of bit string if

[3] The expected standard deviation for the accuracy depends on the samples n in the dataset as $\sigma = 1/(2\sqrt{n})$ (see, for example [BGH+23]); resulting in $\sigma = 0.0005$ for $N_{\text{val}} = 10^6$.

Algorithm 1 GenericPartialDecryption($E_k(\cdot)$)

Input: Oracle access to an r-round block cipher $E_k(\cdot)$, where $P \in \mathcal{P}$, $k \in \mathcal{K}$, and $C \in \mathcal{C}$ such that $C = E_k(P)$.
Output: Neural distinguisher $\mathcal{ND}$ for the highest non-random round.

$\mathcal{ND} \leftarrow$ randomly initialize the neural network (cf. Section 4.4)
$(r_0, \delta_0) \leftarrow$ Evolutionary Optimizer (select starting round r_0 and input difference δ_0, cf. Section 4.3)
▷ Execute curriculum learning starting from round r_0 (cf. Section 4.5)
$A_{\text{val}} \leftarrow 1.0$
$r \leftarrow r_0$
while $A_{\text{val}} > 0.50 + 10\sigma$ **do**
    $\mathcal{D}_{\text{train}} \leftarrow$ GeneratePDDataset($r, N_{\text{train}}, \delta_0, \textit{dataformat}$)
    $\mathcal{D}_{\text{val}} \leftarrow$ GeneratePDDataset($r, N_{\text{val}}, \delta_0, \textit{dataformat}$)
    Train $\mathcal{ND}$ on $\mathcal{D}_{\text{train}}$
    $A_{\text{val}} \leftarrow$ accuracy of $\mathcal{ND}$ on $\mathcal{D}_{\text{val}}$ ▷ Evaluate neural distinguisher
    $r \leftarrow r + 1$ ▷ Progress to next round r
end while
return $\mathcal{ND}$

Algorithm 2 GeneratePDDataset(r, N, δ_0, *dataformat*)

Input: Round r, number of samples N, and input difference δ_0.
Output: Dataset $\mathcal{D}$ with N labeled samples for round r.

$\mathcal{D} \leftarrow \emptyset$
for $j = 1, \ldots, N$ **do**
    $P, k \leftarrow$ random choice of $P \in \mathcal{P}$, $k \in \mathcal{K}$
    $y \leftarrow$ random choice from $\{0, 1\}$ ▷ Randomly choose a label
    $(C, C') \leftarrow (E_k^r(P), E_k^r(P \oplus \delta_0))$ ▷ Generate ciphertext pair
    $(C, C') \leftarrow$ random pair of bit strings if $y = 0$
    $\mathcal{D} \leftarrow \mathcal{D} \cup \{$bit vector according to *dataformat*$\}$ ▷ cf. Section 4.1
end for
return $\mathcal{D}$

$y = 0$. The dataset $\mathcal{D}$ consists of labeled feature vectors according to the chosen data format, which is subsequently used to train and validate the neural distinguisher in the main algorithm.

Our parameter choices follow the ones of [BGH+23]: All our $\mathcal{ND}$s are trained over 40 epochs per round using $N_{\text{train}} = 10^7$ samples, with a batch size of 5000, and validated on $N_{\text{val}} = 10^6$ samples. Therefore, each of the five fresh test sets contains 10^6 samples drawn with equal class priors; the expected imbalance per set is $\leq 0.05\%$. Averaging accuracy over the five sets reduces any residual bias to $\leq 0.02\%$, making the reported mean robust. Our reporting is consistent with prior works such as [BGH+23] and statistically robust. Additionally, we apply the simple polishing step of [BGH+23], where the final network undergoes retraining for three iterations. Each iteration involves repeating the training for one epoch 100 times on 10^7 fresh training samples, resulting in a total training dataset of $100 \times 10^7 = 10^9$ samples per iteration. Since the additional data we

are using, we decide to apply the polish step just for the round attacked and not for every step of the staged training. With a batch size of 10,000, we use the ADAM optimizer, gradually decreasing the constant learning rate at each iteration from 10^{-4} to 10^{-5} and finally to 10^{-6}. Validation is performed on 10^7 samples at each iteration.

Our single-pair classifiers, with $m = 1$, are extrapolated to multi-pair distinguishers ($m = 8$), using the combined response computed as in Section 4.6, to enable fair comparison with the related work. In both cases, the reported accuracies are the average on five fresh testing datasets of size 10^6.

5 Neural Distinguishers via Generic Partial Decryption

In the following, we apply our GENERICPARTIALDECRYPTION (cf. Section 4.7) to SIMON, SIMECK, and ARADI. Summarizing the results discussed in detail below, we find that for SIMON and SIMECK, our generic pipeline achieves competitive accuracies compared to the highly specialized approach of [LLS+24]. Moreover, we can explain the accuracies obtained by [LLS+24] by their distinct contributions: the use of multiple pairs, additional data, and the actual partial decryption.

5.1 Simon and Simeck

We perform the training starting from round 8 to round 12 with input difference: `0x400` and `0x80` respectively for SIMON 32/64 and SIMECK 32/64 (Table 1) (for the choice of the input differences cf. Section 4.3). In each table, we present our results for GENERICPARTIALDECRYPTION applied to SIMON in the DBITNET-columns alongside the results of [LLS+24], which exploit specific features of the SIMON and SIMECK ciphers.

SIMON. For SIMON (Table 1), in the simple data format (C, C'), DBITNET distinguishes better than random up to 11 rounds; in the multi-pair setting ($m = 8$), the 11 rounds accuracy increases by 4 points, and the 12 rounds accuracy remains borderline, at 2.6σ above randomness. Applying the simple polishing step (cf. Section 4.7) brings the accuracy to 0.5097 (19σ above random). These results indicate that a 12-round distinguisher for SIMON can be achieved with the simple data format by combining *i)* multi-pair evaluation and *ii)* a simple polishing step with additional training data.

The remaining gap with the 12-rounds accuracy 0.5152 from [LLS+24] is filled by feature engineering: DBITNET, combined with the GENERICPARTIALDECRYPTION, achieves slightly higher accuracy (0.5153) using the same feature engineering technique, and up to 0.5156 under the (LD, 1PD, 2PD) format.

SIMECK. For SIMECK (Table 1), we observe similar results, unsurprisingly, due to the similarities of the two primitives. However, the key schedule difference between the two primitives does not seem to impact the accuracy of the $\mathcal{ND}$.

Table 1. Results for SIMON 32/64, SIMECK 32/64, and ARADI. SIMON: (†) See Section 4.5; (a) Refer to [LLS+24]; (b) The $\mathcal{ND}$ is the same of the case $m = 1$ but we change the evaluation step according Section 4.6. SIMECK: (†) See Section 4.5; (a) Refer to [LLS+24]; (b) The $\mathcal{ND}$ is the same as the case $m = 1$ but we change the evaluation step according Section 4.6. ARADI: (a) The $\mathcal{ND}$ is the same of the case $m = 1$ but we change the evaluation step according Section 4.6.

	[LLS+24]	**DBitNet**			**DBitNet**			**DBitNet**		
	$(\Delta^r, C, C', \Delta_R^{r-1}, \tilde{\Delta}_R^{r-2})$	(C, C')			$(\Delta^r, C, C', \tilde{\Delta}^{r-1}, \tilde{\Delta}^{r-2})$			$(\Delta^r, \tilde{\Delta}^{r-1}, \tilde{\Delta}^{r-2})$		
SIMON32/64	$m = 8$	$m = 1$	$m = 8$	$m = 8$	$m = 1$	$m = 8$	$m = 8$	$m = 1$	$m = 8$	$m = 8$
Round	adv.	Simple	Simple	Pol.	Simple	Simple	Pol.	Simple	Simple	Pol.
8	-	0.8367	0.9991	-	0.8417	0.9993	-	0.8408	0.9993	-
9	0.9176	0.6556	0.8892	-	0.6584	0.8938	-	0.6580	0.8933	-
10	0.6975	0.5626	0.6761	-	0.5655	0.6859	-	0.5646	0.6827	-
11	0.5609	0.5150	0.5479	-	0.5177	0.5567	-	0.5173	0.5561	-
12	**0.5152**	0.5004	0.5013	0.5097	0.5022	0.5077	**0.5153**	0.5023	0.5064	**0.5156**

	[LLS+24]	**DBitNet**			**DBitNet**			**DBitNet**		
	$(\Delta^r, C, C', \Delta_R^{r-1}, \tilde{\Delta}_R^{r-2})$	(C, C')			$(\Delta^r, C, C', \tilde{\Delta}^{r-1}, \tilde{\Delta}^{r-2})$			$(\Delta^r, \tilde{\Delta}^{r-1}, \tilde{\Delta}^{r-2})$		
SIMECK32/64	$m = 8$	$m = 1$	$m = 8$	$m = 8$	$m = 1$	$m = 8$	$m = 8$	$m = 1$	$m = 8$	$m = 8$
Round	adv.	Simple	Simple	Pol.	Simple	Simple	Pol.	Simple	Simple	Pol.
8	-	0.8470	0.9998	-	0.9022	1.0000	-	0.9025	1.0000	-
9	0.9952	0.6819	0.9276	-	0.7075	0.9562	-	0.7074	0.9564	-
10	0.7354	0.5601	0.6832	-	0.5700	0.7127	-	0.5702	0.7133	-
11	0.5646	0.5133	0.5472	-	0.5168	0.5595	-	0.5170	0.5612	-
12	**0.5146**	0.5007	0.5028	0.5109	0.5026	0.5100	**0.5155**	0.5028	0.5090	**0.5156**

	DBitNet			**DBitNet**			**GohrAMS**			**DBitNet**		
	(C, C')			$(\Delta^r, C, C', \tilde{\Delta}^{r-1}, \tilde{\Delta}^{r-2})$			$(\Delta^r, C, C', \tilde{\Delta}^{r-1}, \tilde{\Delta}^{r-2})$			$(\Delta^r, \tilde{\Delta}^{r-1}, \tilde{\Delta}^{r-2})$		
ARADI	$m = 1$	$m = 8$	$m = 8$	$m = 1$	$m = 8$	$m = 8$	$m = 1$	$m = 8$	$m = 8$	$m = 1$	$m = 8$	$m = 8$
Round	Simple	Simple	Pol.	Simple	Simple	Pol.	Simple	Simple	Pol.	Simple	Simple	Pol.
3	1.0000	1.0000	-	1.0000	1.0000	-	1.0000	1.0000	-	1.0000	1.0000	-
4	1.0000	1.0000	-	1.0000	1.0000	-	1.0000	1.0000	-	1.0000	1.0000	-
5	0.5974	0.7533	0.7634	**0.7732**	**0.9984**	**0.9988**	**0.7733**	**0.9988**	**0.9988**	**0.7721**	**0.9988**	**0.9988**

More importantly, these results show that, contrary to the common-held belief, the presence of the ciphertexts is not always mandatory to reach optimal accuracy; indeed, our best performing format for Simon and Speck, (LD, 1PD, 2PD), only uses differences.

5.2 Aradi

We train the neural distinguisher ($\mathcal{ND}$) for ARADI from round 3 to round 5 using the input difference `0x10000000000000000000000`. The distinguishers (Table 1) reach 100% accuracy on 3 and 4 rounds, while the best differential trail over 4 rounds has probability 2^{-32}. Furthermore, with the input difference we used, the best differential trail has probability 2^{-60} (found using CLAASP [BGG+24]). Another previous work [BFG+24b, Table 10] achieves 5-round accuracy of 0.5954. This accuracy is close to our results in the (C, C') scenario. In the multi-sample and partially decrypted scenario, we significantly surpass these results, reaching accuracies of up to almost perfect accuracy in round 5.

On 5 rounds, the addition of features derived from partial decryption has a considerable positive impact on the accuracy, with DBitNet going from just under 60% accuracy in the (C, C') format to around 77% with the composite format $(\Delta^r, C, C', \tilde{\Delta}^{r-1}, \tilde{\Delta}^{r-2})$. GohrAMS reaches a similar performance. In contrast, the best 5-round differential trail has probability 2^{-50}; if the input difference is restricted to the one used by our distinguisher, the best trail has probability 2^{-94}.

On the other hand, we can observe deterministic truncated differential properties when the input difference is in a single S-Box position. Namely, by construction of the linear layer, such an input difference propagates to 3 non-zero differences after one round with probability 1. At round 2, the truncated propagation is still deterministic, and 9 S-Box positions are active. At round 3, the activity pattern of 22 S-Boxes is non-deterministic, while 10 S-Boxes have a 0 difference with probability 1. This 40-bit property is sufficient to build a strong distinguisher and can be directly checked from the round 4 difference. Indeed, when inverting the last round, the round 4 key addition does not change the difference, and the linear layer propagates it in a deterministic way. The input difference before and after the inversion of S-boxes depends on the round 4 key, but we are only interested in whether they have a zero difference. For bijective S-Boxes, a zero input difference always goes to a 0 output difference and vice versa so that we can observe which S-Box positions are 0 at round 3. Therefore, giving the partial decryption to the neural distinguisher directly gives it a strong property to observe, which it otherwise would have to learn.

6 Key Recovery Attack via Generic Partial Decryption

6.1 Neural Key Recovery in the Literature

In [Goh19], neural distinguishers are used to mount a key recovery attack. The attack divides the cipher into three parts: the Prepended n_{PP} rounds, the Neural Distinguisher on n_{ND} rounds, and the Key Recovery part on n_{KR} rounds.

The Basic Attack. In its most basic form, Gohr's neural key recovery [Goh19] targets $n_{\text{ND}} + n_{\text{KR}}$ rounds. The attacker queries the encryption of plaintext pairs with difference δ. For each candidate subkey in the last n_{KR} rounds, the pairs are decrypted and submitted to the neural distinguisher. Based on the wrong key randomization hypothesis, the distribution induced by incorrect key decryption is assumed to differ from the distribution learned by the neural distinguisher.

Wrong Key Response Profile. Decryption with an incorrect key does not, in practice, induce a random distribution of the pairs for Speck 32. The author of [Goh19] observes that the XOR difference between a wrong key and the actual key influences the scoring output by the neural distinguisher. A *wrong key response profile* quantifies, for each key difference value, the expected distribution of the output of the neural distinguisher. This information helps greatly reduce the number of examined candidate keys, as it selects which direction to walk to get closer to the correct key.

Reaching More Rounds. If there exists an input difference γ, which has probability p to propagate to the $\mathcal{ND}$ input difference δ after n_{PP} rounds, then a pair with input difference γ encrypted for $n_{\text{PP}} + n_{\text{ND}}$ rounds should be classified as real by the n_{ND} rounds neural distinguisher with probability $p \cdot \texttt{TPR} + (1-p) \cdot \texttt{FPR}$, where TPR and FPR denote respectively the true positive and false positive rates of the neural distinguisher. For a random permutation, this probability simply becomes FPR. The number of positive predictions can, therefore, be used as a distinguisher for $n_{\text{PP}} + n_{\text{ND}}$ rounds. However, the number of pairs required to reach over 50% confidence with the above distinguisher grows with the square of the inverse of p; to counter that effect, the author of [Goh19] proposes to use *Probabilistic Neutral Bits* (PNBs) to build structures within which all the pairs are expected to behave similarly in the first n_{PP} rounds.

Definition 3. *(Probabilistic Neutral Bit) Let π_k be a keyed permutation, and $\gamma \to \delta$ be a differential for π_k. A bit i is a* Neutral Bit *for γ, δ if it does not influence the differential. Let x_i^* denote plaintext x with bit position i flipped; if $prob\pi_k(x_i^*) \oplus \pi_k(x_i^* \oplus \gamma) = \delta | \pi_k(x) \oplus \pi_k(x \oplus \gamma) = \delta = 1$, then bit i is neutral. More generally, i is a* Probabilistic Neutral Bit *with probability p if*

$$\mathbb{P}[\pi_k(x_i^*) \oplus \pi_k(x_i^* \oplus \gamma) = \delta | \pi_k(x) \oplus \pi_k(x \oplus \gamma) = \delta] = p.$$

Attacking Large Round Keys. The above attack requires iterating over all possible key error bit vectors. Therefore, little attention has been given to neural key recovery on block ciphers with larger round keys. The main result for that setting [CBSY22] proposes to use different neural distinguishers for different subsets of the key bits. These subsets are identified through the *bit sensitivity test* [CSY23], which identifies *informative bits* of the ciphertext w.r.t the classification by the neural distinguisher. A key recovery using this technique focuses on key bits in the last round key that have a high impact on the corresponding ciphertext bits after partial decryption.

6.2 Neural Key Recovery on Aradi

Aradi uses large (128-bit) round keys. Therefore, we build an alternative key recovery strategy using probabilistic neutral bits in the *decryption direction*. This technique was introduced by Aumasson for differential-linear cryptanalysis against ChaCha [AFK+08]. To distinguish these bits from the neutral bits used in the encryption direction, we call them Probabilistic Neutral Key Bits (PNKBs).

PNKBs and How to Find Them. A bit of the last round key is said to be a PKNB when, if it is incorrectly guessed, it does not change the prediction of the neural distinguisher for the corresponding partially decrypted ciphertext pair. In [AFK+08], probabilistic neutral bits are defined as follows for a function $f(k, W)$ of an unknown partial key k and known information W:

Definition 4. *Probabilistic Neutral Key Bit (PNKB) The neutrality measure of the key bit k_i with respect to the function $f(k, W)$ is defined as γ_i, where*

$$\mathbb{P}[f(k, W) = f(k_i^*, W)] = \frac{1}{2} \cdot (1 + \gamma_i)$$

is the probability (over all k and W) that complementing the key bit k_i does not change the output of $f(k, W)$.

In our analysis, given an $r+1$-round ciphertext pair, we decrypt the last round using a candidate key k and build the candidate decryptions $C_0^* = g_{r+1}^k(C_0)$ and $C_1^* = g_{r+1}^k(C_1)$, adapting the notations introduced in Definition 1; from these, we set $W = (\Delta^{r*}, \tilde{\Delta}^{r-1*}, \tilde{\Delta}^{r-2*})$, and set $f(k, W) = \mathcal{ND}(W)$. A PNKB is a key bit that does not change the output of the neural distinguisher when guessed incorrectly. To test whether a key bit b is a PNKB, we build a dataset $\mathcal{D}_{\text{PNKB}}$, in which the random samples are defined as usual, and the real samples are built from $C_0^* = g_{r+1}^{k^b}(C_0)$ and $C_1^* = g_{r+1}^{k^b}(C_1)$, where k^b is the correct round key k with bit b randomized. If the bit is indeed a PNKB, then the accuracy of the neural distinguisher on this modified dataset is equivalent to its accuracy on a normal r round dataset. Preliminary experiments show that some key bits are naturally almost neutral in our trained $\mathcal{ND}$ for Aradi. However, building a practical attack requires a sufficient number of PNKBs, with neutrality measure as high as possible. We, therefore, retrain our neural distinguishers to reduce their sensitivity to changes in the PNKBs. More specifically, starting from an empty set of PNKBs, and for each last round key bit position i, we retrain our $\mathcal{ND}$ on the corresponding $\mathcal{D}_{\text{PNKB}}$ dataset. If the accuracy remains sufficiently close to the basic $\mathcal{ND}$, we keep bit i as neutral and proceed to the next. If not, bit i is re-set to its correct value, and the next bit is investigated. At the end of this procedure, we have a set of PNKBs, as well as the dual set of important key bits.

A Simple 5-rounds Key Recovery. The simplest version of our attack targets 5 rounds of Aradi and uses a 4-round neural distinguisher, trained on input difference `0x10000000000000000000000`. This neural distinguisher has over 99% accuracy, which enables low data complexity key recoveries. Furthermore, it exhibits a set of 88 PNKBs, for last round key indices 0, 3, 4, 7, 8, 9, 10, 11, 12, 14, 15, 17, 18, 19, 21, 22, 23, 26, 27, 28, 30, 31, 32, 35, 36, 39, 40, 41, 42, 43, 44, 46, 47, 49, 50, 51, 53, 54, 55, 58, 59, 60, 62, 63, 64, 67, 68, 71, 72, 73, 74, 75, 76, 78, 79, 81, 82, 83, 85, 86, 87, 90, 91, 92, 94, 95, 96, 99, 100, 103, 104, 105, 106, 107, 108, 110, 111, 113, 114, 115, 117, 118, 119, 122, 123, 124, 126, 127. In addition, we use a set of 16 rotationally equivalent neural distinguishers for rotations in the weight 1 input difference in the leftmost 16 bits of word x of the state. The distinguishers have all equivalent accuracies and map to equivalent rotated PKNBs. When viewing the cipher state as 4 rows of 32 bits, considered as a left and right part, each composed of 4 rows of 16 bits, a rotation of r positions in the input difference results in a rotation of r positions in the left and right parts independently. The basic key recovery strategy is to fix the 88

PNKBs to a random value and attack the remaining 40 non-neutral key bits. Using 16 neural distinguishers for the corresponding 16 rotated input differences and PNKB sets, we iteratively recover the round 5 subkey.

In our attack, we start by recovering the 40 key bits associated with one of our neural distinguishers. Some of these key bits have a very strong signal in that the scoring output by the neural distinguisher for the corresponding key guesses is consistently high; on the other hand, some key bits do not exhibit such a clear behavior. We, therefore, assign a confidence score to each of the guessed key bits and only consider a bit *valid* when this score is high enough. We then iteratively select a new input difference and its set of key bits to attack. The choice of the next input difference is guided by the amount of corresponding *leftover bits*, bits which are part of the input difference's PNKBs and not part of the valid bits set. To keep the attack practical, we need the set of attacked bits to be as small as possible, so we aim for 20 or fewer leftover bits. The attack continues until the set of valid bits has size 128 (*i.e.,* the entire last round key is recovered), or no new bit is recovered for a threshold number of iterations. The remaining 128 bits of the key can be recovered using a similar strategy with a 3-round neural distinguisher. In order to experimentally validate this practical key recovery, the neural network evaluation for 2^{40} potential key candidates was an obstacle. However, the PNKB sets for different distinguishers overlap so that once a small number of key bits have been recovered, it is usually possible to find an overlapping set with less than 20 key bits to enumerate. We, therefore, assume preliminary knowledge of 30 key bits in the first explored set for practical validation and successfully recover the remaining 98 bits of the round 5 key. This attack was performed 25 times, each time with a fresh random key and a random selection of 30 known key bits, and successfully recovered the entire last round key 21 times, with a maximum data complexity of 520 pairs, slightly over 2^9 chosen plaintexts. More specifically, each enumeration of a non-neutral key bits set is tested against 10 pairs, and the maximum number of iterations of the enumeration algorithm was 52 (with some PNKB sets attacked several times until a high enough confidence is achieved). It is likely that the data complexity and success rate of the attack can be further improved, for instance by using more pairs at each iteration (therefore increasing the confidence in the corresponding guesses and reducing the number of iterations), or with a smarter exploration of the connections between overlapping PNKB sets.

Extending the Attack. Our attack can be extended to 7 rounds, at the cost of non-practical time complexity, by having the distinguisher start from round 2 instead of round 0. This requires retraining a neural distinguisher, as the rotational constants of rounds 2 to 6 are not the same as the ones from rounds 0 to 4. Nonetheless, equivalent distinguishers can be built from round 2, which is consistent with observations on the deterministic propagation of input differences through the cipher. In addition, we observe that due to the SPN structure of Aradi, the accuracy of neural distinguishers and differential propagation are similar regardless of the row of the Hamming weight 1 input difference (when we consider the Aradi state as 4 rows). Furthermore, truncated differences within

a single row share similar properties, meaning that when prepending rounds, we do not need to target a specific bit but a column and benefit from a differential effect. For Aradi, the best 2-round differential trails that end with a single-column output difference have probability 2^{-24}. We focus, without loss of generality, on the differential transition from $(\texttt{0xca10314}, \texttt{0xca10314}, 0, 0)$ to an output difference where a single active S-Box is present in column 15. This truncated differential transition holds with approximate probability 2^{20} (as experimentally verified). We experimentally find a set of 76 neutral bits for this differential transition, which may enable longer key recoveries. For a 7-round attack, however, we can use significantly less. Remember that our 4-round distinguishers have over 99% accuracy, so the response profile for pairs that do not follow the differential is very distinct. Therefore, in the attack, we can build small structures of 2^8 pairs; we need on average 2^{20} structures to find one that follows the 2-round differential. Within each structure, we test one pair at a time by testing the corresponding 40 key-bit candidates; the response profile of the correct structure is expected to be the highest. From there, the attack proceeds as in the previous section, using different neural distinguishers for different key-bit subsets. We expect that with 2^8 pairs per structure, the key recovery procedure will require a single pass for each of the input differences, resulting in a worse-case data complexity of $2^{20} \cdot 2^8 \cdot 16 = 2^{32}$. The time complexity is on average 2^{20} neural distinguisher evaluations per pair, but dominated by the initial 2^{40} neural evaluations for the first PKNB set, yielding a total time complexity of 2^{72} $\mathcal{ND}$ evaluations.

From Neural to Classical Key Recovery. The neural distinguisher findings are helpful in identifying that there is a relevant property for 4 and 5 rounds. However, for cryptographers, knowing there is an attack is not sufficient, and understanding it is crucial. We therefore propose to use the $\mathcal{ND}$ results to mount a classical key recovery. For the 4-round distinguishers, the key property is that a set of 40 bits (10 columns) in round 3 is always 0 due to the slow diffusion of the single active nibble in the input difference. The activity pattern of the S-Boxes is preserved by one round of partial decryption so that decrypting the 4-round pair with a random key is sufficient to observe the 40 zeros pattern for a real pair. On the other hand, for random pairs, it occurs with probability 2^{-40}, so that the corresponding distinguisher is highly accurate: its true positive rate is 1 (as all real pairs follow this pattern), and its false positive rate is $1\text{-}2^{-40}$.

Transforming this distinguisher into a 5-round key recovery requires identifying the key bits involved in the computation of the 40 fixed positions in round 3. Each of these columns depends, by the construction of the linear layer, on 3 columns of the round 4 difference. Since we are only interested in the activity pattern and not the actual value of the S-Box inputs, we do not need the corresponding round 4 key. On the other hand, each of these round 4 columns depends on 3 columns of round 5, for which we do need to obtain the key to get the correct values for the columns of round 3. More specifically, there are 10 sets of 36 key bits, each corresponding to one column, to be recovered.

The probability for an incorrect key guess to result in the expected 4-zeros column pattern is 2^{-4}; after analyzing N pairs, the probability for a wrong key guess to appear for all N pairs is $2^{-4 \cdot N}$. This figure matches the neural key recovery experiments, where 10 pairs were sufficient for each key guessing phase. The data complexity of the corresponding key recovery is 10 pairs, and the time complexity, $2^{36} \cdot 4 \cdot 10$, rounded up to 2^{42}. In comparison, the first step of the neural key recovery of the previous section requires $10 \cdot 2^{40}$ $\mathcal{ND}$ evaluations, with each $\mathcal{ND}$ evaluation accounting for significantly more than a simple operation, resulting in higher time complexity. On the other hand, the neural key recovery may perform similarly, or better, if retrained for the subsets of 36 non-neutral key bits used in the classical attack; however, we did not perform such experiments, as we find the classical attack more satisfying.

7 Conclusion

In this paper, we recognize neural distinguishers as a useful tool for identifying differential properties that do not immediately appear from classical differential characteristics. We, therefore, aim to make their application to new primitives easier, building on the generic AutoND framework and making it more competitive with dedicated approaches. In particular, we notice that recent works, such as [LLS+24], break the classical black-box model (where the neural distinguisher is only given ciphertext pairs) to include information on the structure of the cipher, by including features derived from the deterministic (and sometimes probabilistic) propagation of the difference to the previous rounds. We propose Generic Partial Decryption as a way to include such features automatically, and evaluate the resulting framework on Simon, Simeck, and Aradi. By separating feature engineering from other techniques, such as multiple-pair distinguishers, we observe that the addition of these features is not determinant to the advantage of [LLS+24] over the AutoND framework, but that the multi-pair aspect plays the most important role (even though feature engineering helps fill a small gap between the two techniques). In addition, we propose the first neural key recovery on 5 rounds of Aradi, with practical complexity, and propose an equivalent classical attack, informed by the biases exhibited by the neural distinguisher, with better complexity. In future work, it would be interesting to observe how much similar insight can be leveraged to improve classical attacks. On the neural distinguisher side, it appears that the addition of features sometimes lowers the accuracy, possibly due to larger inputs. Addressing this limitation would help further improve automated neural cryptanalysis, in turn improving our understanding of multiple differential properties.

References

ADG24. Roberto Avanzi, Orr Dunkelman, and Shibam Ghosh. A note on ARADI and LLAMA. *IACR Cryptol. ePrint Arch.*, page 1328, 2024.

AFK+08. Jean-Philippe Aumasson, Simon Fischer, Shahram Khazaei, Willi Meier, and Christian Rechberger. New features of latin dances: Analysis of salsa, chacha, and rumba. In Kaisa Nyberg, editor, *Fast Software Encryption, 15th International Workshop, FSE 2008, Lausanne, Switzerland, February 10-13, 2008, Revised Selected Papers*, volume 5086 of *Lecture Notes in Computer Science*, pages 470–488. Springer, 2008.

BBCD21. Anubhab Baksi, Jakub Breier, Yi Chen, and Xiaoyang Dong. Machine learning assisted differential distinguishers for lightweight ciphers. In *Design, Automation & Test in Europe Conference & Exhibition, DATE 2021, Grenoble, France, February 1-5, 2021*, pages 176–181. IEEE, 2021.

BBCD22. Anubhab Baksi, Jakub Breier, Yi Chen, and Xiaoyang Dong. Machine learning-assisted differential distinguishers for lightweight ciphers. *Classical and Physical Security of Symmetric Key Cryptographic Algorithms*, pages 141–162, 2022.

BDD+24. Christina Boura, Nicolas David, Patrick Derbez, Rachelle Heim Boissier, and María Naya-Plasencia. A generic algorithm for efficient key recovery in differential attacks – and its associated tool. In Marc Joye and Gregor Leander, editors, *Advances in Cryptology – EUROCRYPT 2024*, pages 217–248, Cham, 2024. Springer Nature Switzerland.

BFG+24a. Emanuele Bellini, Mattia Formenti, David Gérault, Juan Grados, Anna Hambitzer, Yun Ju Huang, Paul Huynh, Mohamed Rachidi, Raghvendra Rohit, and Sharwan K. Tiwari. Claasping ARADI: automated analysis of the ARADI block cipher. *IACR Cryptol. ePrint Arch.*, page 1324, 2024.

BFG+24b. Emanuele Bellini, Mattia Formenti, David Gérault, Juan Grados, Anna Hambitzer, Yun Ju Huang, Paul Huynh, Mohamed Rachidi, Raghvendra Rohit, and Sharwan K. Tiwari. CLAASPing ARADI: Automated analysis of the ARADI block cipher. Cryptology ePrint Archive, Paper 2024/1324, 2024.

BG11. Céline Blondeau and Benoît Gérard. Multiple differential cryptanalysis: Theory and practice. In *Fast Software Encryption - 18th International Workshop, FSE 2011*, volume 6733 of *Lecture Notes in Computer Science*, page 35. Springer, 2011.

BGG+24. Emanuele Bellini, David Gerault, Juan Grados, Yun Ju Huang, Rusydi Makarim, Mohamed Rachidi, and Sharwan Tiwari. CLAASP: A cryptographic library for the automated analysis of symmetric primitives. In *Selected Areas in Cryptography SAC 2023: 30th International Conference, Fredericton, Canada, August 1418, 2023, Revised Selected Papers*, page 387408, Berlin, Heidelberg, 2024. Springer-Verlag.

BGH+23. E Bellini, D Gerault, A Hambitzer, M Rossi, et al. A cipher-agnostic neural training pipeline with automated finding of good input differences. *IACR TRANSACTION ON SYMMETRIC CRYPTOLOGY*, 2023(3):184–212, 2023.

BGL+22. Zhenzhen Bao, Jian Guo, Meicheng Liu, Li Ma, and Yi Tu. Enhancing differential-neural cryptanalysis. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 318–347. Springer, 2022.

BGPT21a. Adrien Benamira, David Gérault, Thomas Peyrin, and Quan Quan Tan. A deeper look at machine learning-based cryptanalysis. In Anne Canteaut and François-Xavier Standaert, editors, *Advances in Cryptology - EUROCRYPT 2021 - 40th Annual International Conference on the Theory and*

Applications of Cryptographic Techniques, Zagreb, Croatia, October 17-21, 2021, Proceedings, Part I, volume 12696 of *Lecture Notes in Computer Science*, pages 805–835. Springer, 2021.

BGPT21b. Adrien Benamira, David Gerault, Thomas Peyrin, and Quan Quan Tan. A deeper look at machine learning-based cryptanalysis. In *Advances in Cryptology–EUROCRYPT 2021: 40th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, October 17–21, 2021, Proceedings, Part I 40*, pages 805–835. Springer, 2021.

BLYZ23. Zhenzhen Bao, Jinyu Lu, Yiran Yao, and Liu Zhang. More insight on deep learning-aided cryptanalysis. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 436–467. Springer, 2023.

BR21. Emanuele Bellini and Matteo Rossi. Performance comparison between deep learning-based and conventional cryptographic distinguishers. In *Intelligent Computing: Proceedings of the 2021 Computing Conference, Volume 3*, pages 681–701. Springer, 2021.

BRRT24. Emanuele Bellini, Mohamed Rachidi, Raghvendra Rohit, and Sharwan K. Tiwari. Mind the composition of Toffoli gates: Structural algebraic distinguishers of ARADI. *IACR Cryptol. ePrint Arch.*, page 1559, 2024.

BS91. Eli Biham and Adi Shamir. Differential cryptanalysis of DES-like cryptosystems. *J. Cryptology*, 4:3–72, 1991.

BSS+13. Ray Beaulieu, Douglas Shors, Jason Smith, Stefan Treatman-Clark, Bryan Weeks, and Louis Wingers. The SIMON and SPECK families of lightweight block ciphers. Cryptology ePrint Archive, Paper 2013/404, 2013.

CBSY22. Yi Chen, Zhenzhen Bao, Yantian Shen, and Hongbo Yu. A deep learning aided key recovery framework for large-state block ciphers. *Cryptology ePrint Archive*, 2022.

Cop94. D. Coppersmith. The Data Encryption Standard (DES) and its strength against attacks. *IBM J. Res. Dev.*, 38(3):243250, May 1994.

CSY23. Yi Chen, Yantian Shen, and Hongbo Yu. Neural-aided statistical attack for cryptanalysis. *The Computer Journal*, 66:2480–2498, 2023.

CSYY22. Yi Chen, Yantian Shen, Hongbo Yu, and Sitong Yuan. A New Neural Distinguisher Considering Features Derived From Multiple Ciphertext Pairs. *The Computer Journal*, 03 2022.

DCC23. Haoran Deng, Xianghui Cao, and Yu Cheng. Attention in differential cryptanalysis on lightweight block cipher SPECK. In *2023 20th Annual International Conference on Privacy, Security and Trust (PST)*, pages 1–9. IEEE, 2023.

Dom12. Pedro Domingos. A few useful things to know about machine learning. *Communications of the ACM*, 55(10):78–87, 2012. `https://dl.acm.org/doi/pdf/10.1145/2347736.2347755`.

ERP22. Amirhossein Ebrahimi, Francesco Regazzoni, and Paolo Palmieri. Reducing the cost of machine learning differential attacks using bit selection and a partial ML-distinguisher. In *International Symposium on Foundations and Practice of Security*, pages 123–141. Springer, 2022.

GHHP24. David Gerault, Anna Hambitzer, Moritz Huppert, and Stjepan Picek. Sok: 5 years of neural differential cryptanalysis. *Cryptology ePrint Archive*, 2024.

GLN22a. Aron Gohr, Gregor Leander, and Patrick Neumann. An assessment of differential-neural distinguishers. *Cryptology ePrint Archive*, 2022.

GLN22b. Aron Gohr, Gregor Leander, and Patrick Neumann. An assessment of differential-neural distinguishers. Cryptology ePrint Archive, Paper 2022/1521, 2022. https://eprint.iacr.org/2022/1521.

GLN23. Aron Gohr, Gregor Leander, and Patrick Neumann. An assessment of differential-neural distinguishers. In *AICrypt23 - 3RD Workshop on Artificial Intelligence and Cryptography*, 2023.

GMW24. Patricia Greene, Mark Motley, and Bryan Weeks. ARADI and LLAMA: Low-latency cryptography for memory encryption. Cryptology ePrint Archive, Paper 2024/1240, 2024.

Goh19. Aron Gohr. Improving attacks on round-reduced Speck32/64 using deep learning. In *Advances in Cryptology–CRYPTO 2019: 39th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18–22, 2019, Proceedings, Part II 39*, pages 150–179. Springer, 2019.

HRCF21. ZeZhou Hou, JiongJiong Ren, ShaoZhen Chen, and AnMin Fu. Improve Neural Distinguishers of SIMON and SPECK. *Sec. and Commun. Netw.*, 2021, jan 2021.

LDLS21. Luc Libralesso, François Delobel, Pascal Lafourcade, and Christine Solnon. Automatic Generation of Declarative Models For Differential Cryptanalysis. In Laurent D. Michel, editor, *27th International Conference on Principles and Practice of Constraint Programming, CP 2021, Montpellier, France (Virtual Conference), October 25-29, 2021*, volume 210 of *LIPIcs*, pages 40:1–40:18. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2021.

LLJW21. Zhengbin Liu, Yongqiang Li, Lin Jiao, and Mingsheng Wang. A new method for searching optimal differential and linear trails in ARX ciphers. *IEEE Trans. Inf. Theory*, 67(2):1054–1068, 2021.

LLS+24. Jinyu Lu, Guoqiang Liu, Bing Sun, Chao Li, and Li Liu. Improved (related-key) differential-based neural distinguishers for SIMON and SIMECK block ciphers. *The Computer Journal*, 67(2):537–547, 2024.

LRCL23. JiaShuo Liu, JiongJiong Ren, ShaoZhen Chen, and ManMan Li. Improved neural distinguishers with multi-round and multi-splicing construction. *Journal of Information Security and Applications*, 74:103461, 2023.

RKK19. Sashank J. Reddi, Satyen Kale, and Sanjiv Kumar. On the convergence of adam and beyond, 2019.

RR22. Adrián Ranea and Vincent Rijmen. Characteristic automated search of cryptographic algorithms for distinguishing attacks (CASCADA). *IET Inf. Secur.*, 16(6):470–481, 2022.

SM23. Ayan Sajwan and Girish Mishra. Comparative analysis of resnet and densenet for differential cryptanalysis of SPECK 32/64 lightweight block cipher. In *International Conference on Cryptology & Network Security with Machine Learning*, pages 495–504. Springer, 2023.

SSL+22. Tao Sun, Dongsu Shen, Saiqin Long, Qingyong Deng, and Shiguo Wang. Neural distinguishers on TinyJAMBU-128 and GIFT-64. In *International Conference on Neural Information Processing*, pages 419–431. Springer, 2022.

SWW21. Ling Sun, Wei Wang, and Meiqin Wang. Accelerating the search of differential and linear characteristics with the SAT method. *IACR Trans. Symmetric Cryptol.*, 2021(1):269–315, 2021.

SZM21. Heng-Chuan Su, Xuan-Yong Zhu, and Duan Ming. Polytopic attack on round-reduced Simon32/64 using deep learning. In *Information Security and Cryptology: 16th International Conference, Inscrypt 2020,*

Guangzhou, China, December 11–14, 2020, Revised Selected Papers, pages 3–20. Springer, 2021.

WTZ+22. Huijiao Wang, Jiapeng Tian, Xin Zhang, Yongzhuang Wei, and Hua Jiang. Multiple differential distinguisher of SIMECK32/64 based on deep learning. *Security & Communication Networks*, 2022.

YK21. Tarun Yadav and Manoj Kumar. Differential-ml distinguisher: Machine learning based generic extension for differential cryptanalysis. In *Progress in Cryptology LATINCRYPT 2021: 7th International Conference on Cryptology and Information Security in Latin America, Bogotá, Colombia, October 68, 2021, Proceedings*, page 191212, Berlin, Heidelberg, 2021. Springer-Verlag.

YZS+15. Gangqiang Yang, Bo Zhu, Valentin Suder, Mark D. Aagaard, and Guang Gong. The SIMECK family of lightweight block ciphers. Cryptology ePrint Archive, Paper 2015/612, 2015.

ZC18. Alice Zheng and Amanda Casari. *Feature engineering for machine learning: principles and techniques for data scientists*. " O'Reilly Media, Inc.", 2018.

ZLWL23. Liu Zhang, Jinyu Lu, Zilong Wang, and Chao Li. Improved differential-neural cryptanalysis for round-reduced Simeck32/64. *Frontiers of Computer Science*, 17(6):176817, 2023.

ZWC23. Liu Zhang, Zilong Wang, and Yindong Chen. Improving the accuracy of differential-neural distinguisher for des, chaskey, and present. *IEICE TRANSACTIONS on Information and Systems*, 106:1240–1243, 2023.

ZZY+21. Runlian Zhang, Mi Zhang, Jiaxu Yan, Yixing Li, Xiaonian Wu, and Lingchen Li. Differential cryptanalysis of TweGIFT-128 based on neural network. In *2021 IEEE Sixth International Conference on Data Science in Cyberspace (DSC)*, pages 529–534. IEEE, 2021.

A Useful diagrams of the analyzed ciphers

In this section, we report some useful diagrams that can help visualize the design of the analyzed cipher.

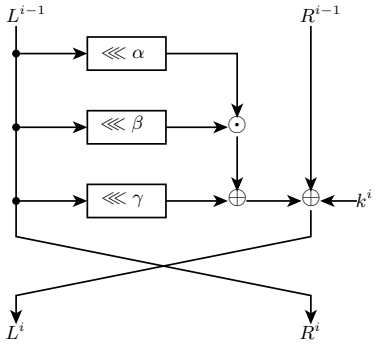

Fig. 1. Round function of Simon and Simeck.

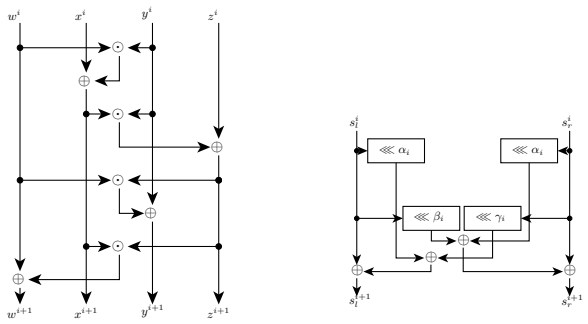

Fig. 2. On the left the S-box π and on the right the component function L_i of the linear layer Λ_i in ARADI.

Table 2. Constants used in ARADI.

$i \mod 4$	α_i	β_i	γ_i
0	11	8	14
1	10	9	11
2	9	4	14
3	8	9	7